LUNAR-ROVER / DRAKE / ACROBOT

Current iLQR implementation and the path to real-time control

Code review and engineering proposal 1 November 2026

The main distinction

The controller replans every 10 ms of *simulated* time, but it waits for a complete optimization before advancing the plant. Receding-horizon control is implemented; meeting a 10 ms wall-clock deadline is not. Compiling the code would help performance, but a real-time design also needs bounded work, latency handling and an independent feedback path.

1 What is running today

1.1 Files and responsibilities

Paths below are relative to `/home/rogerio/Projects/lunar-rover/drake/acrobot/`. This document describes the inspected source, not a proposed replacement already implemented.

File	Responsibility
`acrobot_test.py`	YAML/CLI resolution, Drake simulation, controller selection and synchronous execution.
`lqr.py`	Wrapped angle error, torque clipping, finite LQR and iterative infinite-horizon DARE solution.
`ilqr.py`	Nonlinear RK4 model, iLQR optimization, regularization, line search and receding controller.
`trajectory_seed.py`	Direct-collocation initialization and conversion to a sample-and-hold trajectory.
`lqr.yaml`	Controller mode, costs, initial state, duration, constraints and solver settings.

1.2 Current configuration and the three clocks

The YAML currently selects `ilqr_receding`, $N = 300$ controls, $\Delta t = 0.01$ s, 300 execution steps, $Q = \mathrm{diag}(10, 10, 1, 1)$ and $R = 0.01$. Clipping is disabled; the stored 3 N m limit is therefore inactive. Initialization is `auto`; each solve permits at most 50 iterations, with tolerance 10^{-6} and initial regularization 10^{-5}. Both damping scales are 1.

The current initial state is $[0, 0, 0, 0]$: the hanging-down equilibrium. The upright target is $x^\star = [\pi, 0, 0, 0]$. These angles are in radians and the last two components are in rad/s. This is a *full swing-up* request, not the earlier recovery test from $[2.5, 0.6, 0, 0]$.

- **Control sample:** 0.01 simulated seconds between applied commands.
- **Prediction window:** $N\Delta t = 3$ seconds into the future.
- **Execution window:** $300\Delta t = 3$ simulated seconds in total.

None specifies how long the solver takes on the CPU. YAML values override built-in defaults; explicit function or CLI arguments override YAML. Changing the horizon does not extend execution in receding mode; changing `steps` does.

1.3 Controller modes are different algorithms

Mode	Planning	Execution
`infinite`	One upright linearization; iterate DARE to a constant gain.	Apply the same gain for `steps`.
`finite`	One linear model; backward finite-horizon Riccati sweep.	Apply a gain schedule; stop at horizon.
`receding`	Fixed-model finite LQR; compute its first gain once.	Reuse that gain; no nonlinear replanning.
`ilqr`	Optimize one nonlinear finite trajectory.	Track it with feedback; stop at horizon.
`ilqr_receding`	Re-optimize from every measured state.	Apply only the first torque; run for `steps`.

The linear modes use forward Euler discretization. The nonlinear modes use RK4. Comparing them can therefore change both the optimizer and its prediction model.

2 The nonlinear optimization being solved

2.1 Dynamics, error and objective

The state is $x = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^T$ and the scalar input u is elbow torque. Drake evaluates the continuous dynamics $\dot{x} = f(x, u)$. The iLQR prediction model is

$$x_{k+1} = F_{\Delta t}(x_k, u_k), \qquad u_k \text{ held constant during the interval.} \tag{2.1}$$

`nonlinear_step` integrates this map with RK4 and $m = \max(1, \lceil \Delta t/0.01 \rceil)$ internal steps. At the current 10 ms sample, one map evaluation calls the continuous dynamics four times. Drake's simulator evolves the actual plant separately; it is not the RK4 prediction model itself.

Angular deviations use

$$e(x) = \begin{bmatrix} \text{wrap}(\theta_1 - \pi) & \text{wrap}(\theta_2) & \dot{\theta}_1 & \dot{\theta}_2 \end{bmatrix}^T, \tag{2.2}$$

with angular errors in $[-\pi, \pi)$. The optimized discrete objective is

$$J = \frac{1}{2} \sum_{k=0}^{N-1} \left(e_k^T Q e_k + R u_k^2 \right) + \frac{1}{2} e_N^T Q_f e_N. \tag{2.3}$$

There is no explicit Δt factor in the running sum. Costs are discrete weights, so changing the grid requires attention to running/terminal weighting. The default Q_f is a local DARE value matrix for the RK4 model at upright; Q is the fallback if a stabilizing local solution is unavailable. The receding wrapper caches this terminal matrix after its first solve.

Wrapped error is not globally smooth at $\pm\pi$. At the current hanging-down state, the shoulder error is exactly at that branch boundary. Repeated linearization does not remove this discontinuity or guarantee a global swing-up solution.

2.2 What one iLQR iteration does

For a nominal trajectory $(\bar{x}_k, \bar{u}_k)$, the algorithm computes

$$A_k = \left. \frac{\partial F}{\partial x} \right|_{\bar{x}_k, \bar{u}_k}, \qquad B_k = \left. \frac{\partial F}{\partial u} \right|_{\bar{x}_k, \bar{u}_k}. \tag{2.4}$$

These are central finite differences of the *discrete RK4 map*, with perturbation 10^{-5}. They are not repeated upright-only linearizations. A backward dynamic-programming pass builds a local quadratic value model and returns feedforward increments d_k and feedback gains K_k.

A forward trial uses

$$u_k^{\text{trial}} = \bar{u}_k + \alpha d_k - K_k \operatorname{err}(x_k^{\text{trial}}, \bar{x}_k). \tag{2.5}$$

The line search tries $\alpha \in \{1, 1/2, 1/4, 1/8, 1/16, 1/32, 1/64\}$, accepting only finite, lower-cost nonlinear rollouts. Only the feedforward increment is scaled by α. Accepted controls and states become the next nominal trajectory.

The algorithm stops when the control increment is sufficiently small at base regularization, the relative cost improvement is sufficiently small, or an iteration/numerical limit is reached. Cost convergence does not establish terminal accuracy, robustness, constraint feasibility on hardware, or stability of the actual closed loop.

3 Regularization, constraints and initialization

3.1 Regularization is numerical damping

Using V_x, V_{xx} for the downstream value derivatives, the unregularized control terms include

$$q_u = R\bar{u} + B^T V_x, \qquad q_{uu} = R + B^T V_{xx} B, \tag{3.1}$$

$$q_{ux} = B^T V_{xx} A. \tag{3.2}$$

The backward solve uses

$$\widetilde{V}_{xx} = V_{xx} + \lambda s_x I, \tag{3.3}$$

$$h_{uu} = R + B^T \widetilde{V}_{xx} B + \lambda s_u, \qquad h_{ux} = B^T \widetilde{V}_{xx} A, \tag{3.4}$$

$$d = -q_u / h_{uu}, \qquad K = h_{ux} / h_{uu}. \tag{3.5}$$

Here s_x and s_u are `ilqr_state_damping` and `ilqr_control_damping`. They do not add physical joint friction or modify the configured state-cost matrix Q. They add curvature to the local quadratic control problem. The code propagates the original quadratic objective under the resulting damped policy.

Failed backward passes or rejected line searches increase λ by a factor of 10. Accepted steps decrease it by 10, down to its configured minimum. A limit above 10^{12} terminates the numerical recovery. This data-dependent work contributes to runtime variation.

3.2 Torque handling and failure behavior

When clipping is enabled, the scalar feedforward increment is limited so the nominal torque stays within $[-u_{\max}, u_{\max}]$. Feedback is set to zero for a saturated local update, and rollout torques are clipped as well. With one actuator this is a simple local treatment of box bounds; there are no joint-position, velocity, torque-rate or actuator dynamics constraints.

A nonconverged solve may still produce a finite accepted plan, which the current controller executes while reporting its status. If no finite initial rollout exists, it raises an error. There is no deadline-driven fallback, plan-age check or independent actuator loop today.

3.3 How the first plan is initialized

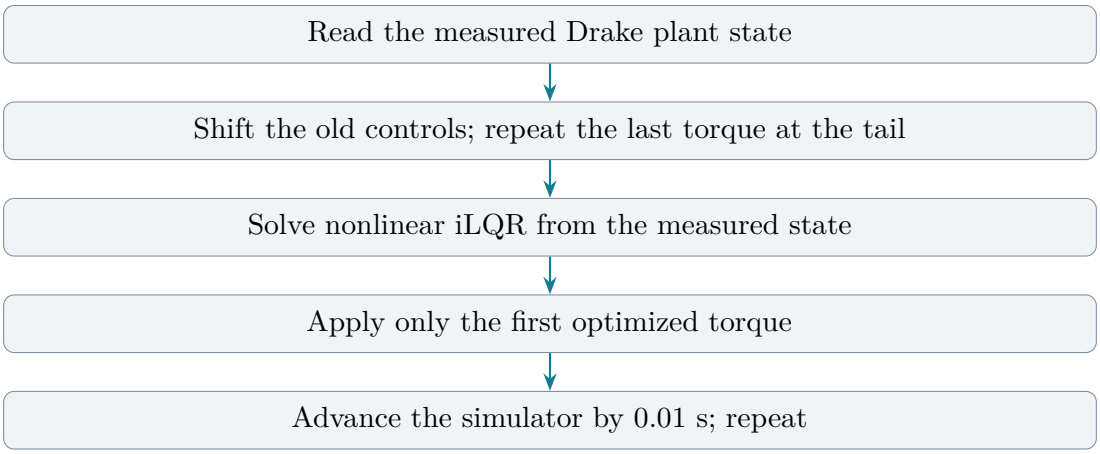
Without explicit initial controls, the optimizer considers zero torque and a local infinite-horizon LQR feedback rollout. In `auto` mode it also tries direct collocation when either initial angular error exceeds 0.25 rad. Forced `collocation` always attempts that candidate; `lqr` omits it.

Collocation fixes the initial and final states, selecting the closest periodic upright representative. For the current 3 s horizon it uses 61 knots spaced by 0.05 s, and solves a nonlinear program through Drake's `Solve`; the source does not fix a particular solver. Knot count is capped at 101. The seed objective uses a torque penalty $\max(1, R)$ to improve conditioning, whereas the subsequent iLQR objective uses the actual configured R.

The continuous collocation trajectory is converted to sample-and-hold RK4 controls using midpoint input samples and time-varying tracking feedback. Directly sampling its torques can fail to track its states. The lowest-cost finite candidate initializes iLQR. `auto` warns and falls back if collocation fails; forced collocation reports an error. This process improves initialization but is not a guarantee of global feasibility or convergence.

4 What receding horizon means in this code

4.1 The actual call sequence



The first solve has no previous controls and uses the initialization process above. Later solves receive a shifted warm start. Every solve rolls out again from its measured initial state. `horizon` remains fixed; execution can exceed that horizon.

In receding mode, `command` returns the first optimized torque. It does *not* run an independent, fast trajectory-feedback loop while optimization is in progress. In contrast, finite `ilqr` explicitly applies the stored trajectory gains during execution.

4.2 The key timing limitation

The state is read, optimization completes, and then `AdvanceTo` runs. The simulated plant effectively waits during the solve. A physical Acrobot would keep moving. Therefore a command computed from a measurement taken hundreds of milliseconds earlier may be obsolete, even if its prediction model was accurate when the solve began.

A wall-clock scheduler or setting a simulator real-time rate would not make a slow optimization meet its deadline. The controller architecture must explicitly handle elapsed time, state-estimation age and delayed plan publication.

5 Why the computation is expensive

5.1 A workload estimate from the source

For four states and one input, central differences require $2(4+1) = 10$ discrete map evaluations per trajectory stage. At $N = 300$ and one RK4 internal step:

$$10 \times 300 \times 4 = 12{,}000 \quad \text{continuous dynamics evaluations per backward pass.} \tag{5.1}$$

This excludes costs, small matrix operations and Python/Drake call overhead. Each trial rollout adds $300 \times 4 = 1{,}200$ continuous evaluations. The implementation also recomputes a final backward pass to provide gains on the accepted trajectory, even though the receding wrapper only applies its first torque.

If I iterations all reach line search, L_i candidates are tested at iteration i, and one initial rollout is counted, a useful estimate is

$$n_f \approx 4mN \left[10(I+1) + 1 + \sum_{i=1}^{I} L_i\right]. \tag{5.2}$$

For two iterations and one successful first trial each, this gives approximately 39,600 continuous evaluations per replan. Startup seeding, rejected backward passes and additional seed rollouts add work. This is an operation-count estimate, **not a measured execution time or speedup claim**.

5.2 Other avoidable costs

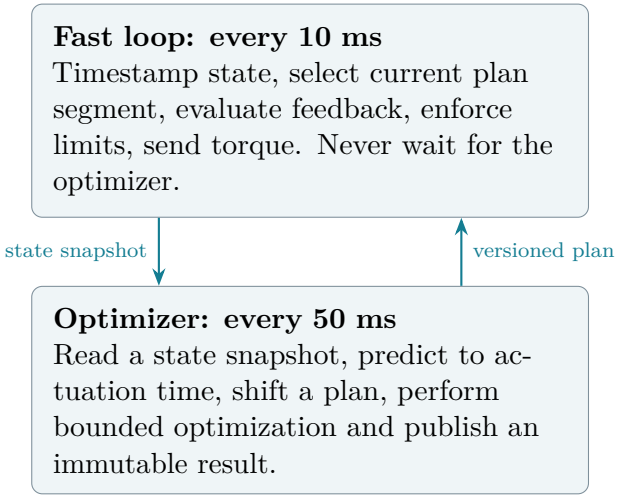
Python loops orchestrate every stage. Each new solve creates a fresh dynamics context; trajectory arrays, derivative lists and trial buffers are allocated repeatedly. Rejected line searches repeat nonlinear integration, and regularization retries recompute Jacobians even when the nominal trajectory has not changed. Printing and visualization share the simulation process.

Coarsening the prediction grid is not automatically a proportional speedup. Keeping a 3 s window but replacing 300 intervals of 0.01 s with 60 intervals of 0.05 s still uses five RK4 internal steps per interval. Thus mN remains 300; integration work can remain similar while stage-level Riccati overhead decreases. Move blocking, nonuniform grids, different sensitivity propagation or a validated integration change are needed to reduce that cost further.

6 A practical real-time architecture

6.1 Separate fast feedback from slower optimization

An initial design target is a 100 Hz actuator/feedback loop and a 20 Hz optimizer. These rates are **proposed targets**, not measured capabilities. Increase optimization frequency only after profiling shows adequate headroom.



Between replans, the fast loop can use

$$u(t) = \mathrm{clip}(\bar{u}_j - K_j \,\mathrm{err}(\hat{x}(t), \bar{x}_j)) \,, \tag{6.1}$$

where j is selected by elapsed plan time, not by how often the optimizer finishes. The published plan needs nominal states, controls, gains, validity timestamps and a version. Reject stale or out-of-order results. Shift warm starts by the number of elapsed prediction intervals; the present one-control shift assumes one solve per plant sample.

If a new plan is unavailable, continue a still-valid previous policy for a bounded age. Near upright, a validated local balancing controller may be a fallback. Outside its tested region, use a defined bounded recovery or stop policy suitable for the hardware. Zero torque and upright LQR are not universally safe fallbacks for arbitrary swing-up states.

6.2 Define deadlines before optimizing code

A 10 ms fast-loop period must include sensing, estimation, scheduling, feedback, communication and actuation. For illustration, reserving 2 ms for state/IO, 1 ms for policy evaluation and 2 ms of margin leaves 5 ms unassigned; it is a design budget, not a benchmark. The 20 Hz optimizer has a separate budget below 50 ms, with margin for publishing and latency.

Bound iteration count, line-search trials and integration work. Check a monotonic-clock deadline inside solver phases, but recognize that cooperative checks cannot interrupt a long blocking kernel or nonlinear-program solve. The fast loop must remain independent even if the worker overruns. Hard real-time claims additionally require bounded lower-level operations and suitable OS scheduling, not just a timeout flag.

7 Changes to implement, in order

7.1 Measure and remove redundant work

Instrument initialization, rollout, linearization, Riccati sweep, line search, final gain computation and total replan latency with a monotonic clock. Record first solve separately from warmed solves. Log iterations, trial counts, regularization, state error and deadline misses without synchronous console IO in the fast path.

Reuse plant contexts and preallocate arrays. Cache derivatives while retrying damping on an unchanged nominal trajectory. Avoid an unused final gain pass in synchronous first-control-only

operation; for the proposed fast feedback loop, retain or compute gains on the final published nominal trajectory. A stale gain schedule must not be advertised as a validated final policy.

7.2 Replace finite differences with model sensitivities

Differentiate the same discrete integration map used for prediction. Drake supports automatic differentiation of dynamical-system operations [1]. A candidate implementation uses an AutoDiff-compatible plant, seeds derivatives with respect to all five components of (x, u), propagates them through RK4 and extracts a 4×5 Jacobian.

The current `float(u)` and `np.asarray(..., dtype=float)` conversions discard derivative information. An AutoDiff path must remove or specialize those casts; merely replacing the finite-difference loop is insufficient. Keep a float path for rollout and validate both paths against central differences over representative states and torques. Analytic or generated integration sensitivities are alternatives. AutoDiff improves derivative evaluation, but is not by itself a latency guarantee.

7.3 Bound optimization and compile the critical path

Use a warm-started, fixed-work online update, initially testing one or two backward/forward iterations per replan and a small bounded line search. Limited iterations can worsen recovery, so they need closed-loop tests and a valid prior plan. This is an engineering approximation; simply capping this iLQR loop is not identical to the formal SQP real-time-iteration algorithm.

Move rollout, sensitivities and Riccati kernels into a persistent C++/Eigen solver with fixed-size matrices and preallocated workspaces. Keep Python for experiment orchestration, configuration and plotting. NumPy operations are already native in part; compiling only a wrapper will leave Python loops and repeated Python-to-Drake calls. Numba cannot automatically compile arbitrary `pydrake` calls.

An alternative is to reformulate the problem in a generated optimal-control solver. acados documents preparation/feedback RTI phases [2] and generation of self-contained C solver code [3]. This is a migration requiring matching dynamics, discretization, costs, bounds and warm starts, not a drop-in switch for the existing Drake plant.

7.4 Separate prediction, control and startup grids

Preserve the fast sample period while evaluating a shorter or nonuniform optimization grid, or hold groups of optimization controls together. A coarser optimizer must still predict fine-loop actuation correctly. If weights represent a continuous-time running cost, integrate or scale them consistently by interval duration and recompute terminal ingredients; do not reuse the old discrete weights blindly.

Keep expensive collocation initialization outside the deadline-critical path. Prepare a feasible maneuver before enabling control, store a small tested trajectory library, or initialize in a worker while a separate policy controls the plant. If no plan is ready, remain in a defined initialization/recovery mode. Starting from hanging-down at the angular branch boundary requires its own validation; the earlier offset-state recovery does not validate full swing-up.

8 Implementation and acceptance plan

8.1 Concrete code changes

Component	Proposed change; not currently implemented
`ilqr.py`	Persistent solver workspace, derivative interface, cached Jacobians, bounded iterations/trials, phase timings and deadline checks.
Receding controller	Timestamped policies with states/controls/gains; elapsed-time warm starts; plan-age validation; explicit solver failure status.
`acrobot_test.py`	Separate feedback and optimization schedules; measurement-to-actuation latency tests; background logging and visualization.
Seed initializer	Startup/offline initialization with explicit success and availability conditions.
`lqr.yaml`	Separate control and prediction intervals, replan period, solver budget, plan-age limit and fallback selection.
Tests / benchmark	Derivative parity, model rollout parity, delayed plans, dropped results, deadline misses, actuator bounds and measured convergence.

The proposed YAML additions are a design sketch: these extra YAML keys do not exist yet and the current parser would reject them.

8.2 What must be demonstrated

1. **Numerical agreement:** compiled and reference rollouts, Jacobians, costs and gains agree within declared tolerances. Test near upright, large deviations and the angle branch boundary.
2. **Closed-loop behavior:** test hanging-down swing-up separately from recovery at $[2.5, 0.6, 0, 0]$ and small disturbances. Record angle and velocity errors separately; their combined norm mixes units.
3. **Realistic limits:** evaluate amplitude and rate bounds, delay, estimation error and disturbances. A trajectory requiring 60 N m does not establish feasibility at 3 N m.
4. **Timing:** on the intended CPU and software stack, measure startup time, warmed median, p95, p99, maximum observed latency, jitter, plan age and deadline-miss rate under representative load. Average runtime alone is insufficient; an observed maximum is not a proven worst-case bound.
5. **Overrun response:** delay the worker, force solver failure and drop a plan. The fast loop must keep its schedule, reject invalid plans and use the declared fallback.

Do not claim 100 Hz real-time control until the complete measurement-to-actuation path meets its budget with the required reliability. Faster simulation playback, smaller tolerances, a lower iteration count or a C++ build alone do not establish that result.

8.3 What has and has not been verified

An earlier session run from $[2.5, 0.6, 0, 0]$ with $R = 0.01$, clipping disabled and the collocation seed produced 300 converged replans. At 3 s its state was approximately

$$[3.133947,\ -0.025649,\ 0.008506,\ 0.071704].$$

The shoulder and elbow angle errors were approximately 0.44° and 1.47°, with peak torque about 59.9 N m. This showed recovery toward upright, not zero terminal error or validation

at a 3 N m bound. Nineteen tests passed during that implementation session. These are historical functional results, not fresh timing measurements for this document.

The current YAML was subsequently changed to hanging-down $[0, 0, 0, 0]$. This PDF does not claim that configuration has been validated, and no new closed-loop or wall-clock benchmark was run for this document. The implementation has neither a deadline-aware online solver nor an independent fast control loop today.

8.4 Commands for reproducing experiments

Run from the host; the relevant dependencies are inside the Drake container. The modular entry point differs from the legacy `drake/acrobot_test.py`.

```
docker exec -it lunar-rover-jazzy-drake fish -c \
 'cd ~/drake; python acrobot/acrobot_test.py'

# Functional tests, not a latency benchmark:
docker exec lunar-rover-jazzy-drake fish -c \
 'cd ~/drake; python -m unittest discover \
  -s acrobot -p "test_*.py"'
```

To reproduce the earlier recovery state, append `-initial-state 2.5 0.6 0 0`; `-steps 600` allows six simulated seconds. Use `-headless -no-wait` for unattended simulation.

Suggested first milestone: add phase timing and persistent workspaces, then validate a derivative implementation. Only after those measurements choose a prediction grid and iteration budget, implement the independent feedback loop, and verify deadlines with realistic constraints.

9 Sources and scope

9.1 Primary source: the inspected project

The primary sources are the five working-tree files in Section 1, inspected on 1 November 2026. This document is a snapshot, not a release specification. Producing it did not change controller code or YAML.

9.2 External references for proposed techniques

[1] Drake, *Basics of Automatic Differentiation*. Documentation on differentiating operations and dynamical systems.
`https://drake.mit.edu/tutorials/autodiff_basics.html`

[2] acados, *Features by Example*. RTI preparation/feedback phases, nonuniform-grid examples and timeout support.
`https://docs.acados.org/features/index.html`

[3] acados, *Python Interface*. Description of generated self-contained C solver code.
`https://docs.acados.org/python_interface/`

References accessed on 1 November 2026. They support the feasibility of implementation approaches, not a specific Acrobot timing claim. RTI/SQP and this iLQR implementation use different solver formulations and must not be treated as interchangeable.